

Algorytmy i Złożoność Obliczeniowa

Projekt 1 – (10 marca 2026)

mgr inż. Damian Mroziński

Zakres projektu

C/C++, algorytmy sortowania, analiza wyników, praca na plikach, skryptowanie

Jakie są cele projektu?

- Zapoznanie z różnymi algorytmami sortowania, ich implementacja oraz analiza efektywności.
- Nabycie umiejętności dopasowania programu do wymagań / ręczne dołączanie bibliotek.
- Nauczenie się podstawowego skryptowania.
- Mądre planowanie projektu, aby nie komplikować sobie życia.

TL;DR

Należy samodzielnie zaimplementować i przeprowadzić analizę określonych algorytmów sortowania (zależnych od planowanej oceny). Jako sortowanie rozumie się uporządkowanie elementów rosnąco. Nie można używać gotowych rozwiązań, również w zakresie struktur danych (typu `std::vector`, `std::list`). Jeśli takie rozwiązania są potrzebne, trzeba je stworzyć samemu, od zera, z ręcznym zarządzaniem pamięcią.

Założenia projektu

1. Sortowanie to uporządkowanie elementów rosnąco.
2. Podstawowym elementem sortowanych struktur jest 4-bajtowa liczba całkowitoliczbowa ze znakiem, tj. `int`.
3. Wszystkie użyte struktury danych powinny być zaimplementowane ręcznie oraz alokowane i zwalniane dynamicznie (zgodnie z badanym rozmiarem tablicy).
4. Obowiązuje C++ w wersji 17.
5. Do kompilacji należy przygotować plik `CMakeLists.txt`. Należy wyróżnić lokalizację z kodem, z bibliotekami oraz z nagłówkami. Dołączanie lokalizacji to robota `CMakeLists.txt`. Nie należy skakać po lokalizacjach przy pisaniu `#include ...`

6. Program musi kompilować się **bez ostrzeżeń** (tak, ostrzeżeń!).

Wystarczy dodać podstawowy zestaw flag do kompilatora

-Wall -Wextra -Werror.

7. Program nie może zostać przerwany błędem. Używanie `try..catch` do obsługi błędów typowych/przewidywalnych nie jest dopuszczalne.
8. Należy przeprowadzić weryfikację poprawności sortowania. Najlepiej w formie prostej funkcji, która sprawdzi, czy elementy są rosnące (dla małych zbiorów można zrobić wizualnie).
9. Pojedynczy pomiar jest niemiernodajny. Aby badania zostały wykonane prawidłowo, należy wywołać każdy przypadek wielokrotnie (np 50 razy), a wyniki uśrednić. Należy również sprawdzić minimum oraz maksimum.
10. Wszystkie wyniki przeprowadzonych badań należy zachować, najprościej dopisywać je jako kolejne wiersze pliku .csv, zapisując zarówno datę i godzinę wykonania badania, wszystkie parametry programu oraz sam czas.
11. Czas badania należy zmierzyć w mikrosekundach za pomocą `std::chrono`. Mierzony czas dotyczy wyłącznie sortowania i nie wlicza się do niego ładowanie danych z pliku, losowanie wartości, czy zapis wyników do pliku.
12. Należy zachować spójność badań, aby można było sensownie porównywać wyniki.
13. Przy badaniach należy losować wartości z pełnego zakresu wybranego typu. Proszę faktycznie pomyśleć co robicie, zwłaszcza w przypadku liczb zmiennoprzecinkowych.
14. Kod musi być sformatowany spójnie i zawierać komentarze.
15. Program powinien wykorzystywać szablony, aby łatwo można było wykorzystać zaimplementowane algorytmy do sortowania różnych struktur i/lub elementy różnych typów.

***NAPRAWDĘ WARTO!** Wymagane na 5.0, ale przyda się każdemu.*

16. Plik z danymi wejściowymi zawsze jest zbudowany w ten sam sposób. W pierwszej linii znajduje się liczba danych do odczytu (np 5), a kolejne linie (w tym przypadku 5 kolejnych linii), zawiera wartości, które należy wczytać do programu.

```
5
76
-142
0
2138
-4
```

W analogiczny sposób należy zapisać posortowane wartości do pliku (jeśli potrzeba).

17. Program powinien posiadać trzy tryby działania.

- a) **Tryb pojedynczego testu**, gdzie następuje sortowanie danych z podanego pliku (z ewentualnym zapisem do pliku wyjściowego) w dowolnym zaimplementowanym algorytmie i z użyciem dowolnej zaimplementowanej struktury danych.
- b) **Tryb badań**, gdzie parametry określają środowisko (m.in. wybrany algorytm, plik wyjściowy z wynikami, liczba powtórzeń, typ danych...).
- c) **Tryb pomocy**, gdzie wyświetlana jest instrukcja użycia programu.

Jeśli, z jakiegoś powodu jest potrzeba innych trybów, proszę je zaimplementować i opisać w helpie.

18. Sterowanie programem odbywa się przy użyciu argumentów do metody głównej (*Typ pracy, plik wejściowy, wielkość instancji, plik wyjściowy, typ danych i tak dalej*). Do ładowania parametrów należy użyć biblioteki udostępnionej przez prowadzącego. **NIE można używać innych parametrów, jeśli czegoś brakuje, proszę założyć issue w repozytorium.**

Sprawdzanie poprawności sortowania będzie częściowo odbywać się skryptami, stąd ściśle wymagania do obsługi parametrów wejściowych.

19. Repozytorium z biblioteką do parametrów oraz przykładowym testowym użyciem.
<https://gitlab.com/mrozo94/for-students>

20. Środowisko, którego będę używał do sprawdzania prac i na którym powinno się wszystko kompilować:

- Kompilator gcc 15.2.0 (*paczka gotowa przez apt*).
- System automatycznego budowania cmake 3.31.6 (*paczka gotowa przez apt*).
- System Xubuntu (25.10) wirtualizowany za pomocą VirtualBox (7.2.4) - *tak naprawdę to akurat nie ma takiego znaczenia.*

Opisy badań

Badanie α - Wpływ parametrów algorytmu.

Badanie polega na sprawdzeniu jak parametry algorytmu wpływają na wyniki.
Należy sprawdzić środkową wartość liczebności zbioru, który będzie użyty w **Badaniu A**.
Zwycięskich parametrów należy użyć w kolejnych badaniach.

W praktyce to nie takie proste, dostrajanie parametrów algorytmów realizuje się w pewnej pętli, powtarzając badania i testując zmiany w różnych instancjach. Algorytm dostrojony do poprawnego działania w małych instancjach niekoniecznie będzie dobrze działał dla dużych instancji i odwrotnie. Tutaj upraszczamy trochę całość tego procesu.

Badanie A - Wpływ liczebności zbioru.

Badanie polega na sprawdzeniu jak liczebność zbioru wpływa na wyniki algorytmu.
Należy sprawdzić minimum cztery zauważalnie różne przypadki, na przykład: 5,10,25,50 tysięcy, bądź 10,25,50,100 tysięcy elementów (w zależności od możliwości sprzętu).

Badanie B - Wpływ rozkładu elementów.

Badanie polega na sprawdzeniu jak rozłożenie elementów wpływa na wynik algorytmu.
Należy, dla wybranego przebadanego rozmiaru z **Badania A** zbadać cztery przypadki: elementy rozrzucone losowo, posortowane malejąco, posortowane rosnąco, posortowane rosnąco w 50%.

Sortowanie wstępne nie jest brane pod uwagę do czasu działania algorytmu (można do niego użyć gotowych funkcji, np `std::sort`).

Badanie C - Wpływ typu danych.

Badanie polega na sprawdzeniu jak typ danych wpływa na wynik algorytmu.
Należy, dla tego samego rozmiaru jak w **Badaniu B** przebadać kilka typów danych, podstawowy (tj. `int`) oraz inne, wybrane.

Badanie Ω - Użycie nieliniowych struktur danych.

Badanie polega na sprawdzeniu jak sprawdzi się nieliniowa struktura danych w porównaniu do struktur liniowych.

Ocenianie - wymagane algorytmy i dodatkowe zadania

Ocena wyjściowa 3.0

- **Algorytmy:** Sortowanie koktajlowe, przez scalanie, przez wstawianie
- **Struktury danych:** Tablica oraz lista jednokierunkowa.
- **Badania:**
 - A - dla każdej kombinacji algorytm + struktura danych.
 - B - dla jednego wybranego algorytmu, dla obu struktur.
 - C - dla jednego wybranego algorytmu, dla jednej struktury.
Do badania pozostałych typów należy użyć: typu zmiennoprzecinkowego, typu bez znaku.
- **Pozostałe:**
 - Obiektość NIE jest wymagana.
 - Repozytorium NIE jest wymagane.
 - Szablony NIE są wymagane.

Ocena wyjściowa 4.0

- **Algorytmy:** Sortowanie kubełkowe, szybkie oraz (do wyboru) dowolny z oceny 3.0 - łącznie trzy.
- **Struktury danych:** Tablica, lista jedno i dwukierunkowa - łącznie trzy.
- **Badania:**
 - A - dla każdej kombinacji algorytm + struktura danych.
 - B - dla jednego wybranego algorytmu, dla dwóch struktur.
 - C - dla jednego wybranego algorytmu, dla jednej struktury.
Do badania pozostałych typów należy użyć: typu zmiennoprzecinkowego, typu bez znaku, typu znakowego `char` (ograniczone do wartości możliwych do zapisania w pliku, tzw. *printables*).
 - Ω - dla jednego wybranego algorytmu (jednego przypadku), należy porównać wszystkie trzy struktury liniowe oraz (do wyboru) kolejkę lub stos.
- **Pozostałe:**
 - Obiektość JEST wymagana.
 - Repozytorium JEST wymagane.
 - Szablony NIE są wymagane.

Ocena wyjściowa 5.0

- **Algorytmy:** Sortowanie kubełkowe, szybkie, shella - łącznie trzy.
- **Struktury danych:** Tablica, lista jedno i dwukierunkowa - łącznie trzy.

- **Badania:**

- α - dla algorytmu shella, dwa różne wzory na odstęp,
dla algorytmu szybkiego trzy wersje pivota: losowy, środkowy, skrajny.
- A** - dla każdej kombinacji algorytm + struktura danych.
- B** - dla jednego wybranego algorytmu, dla trzech struktur.
- C** - dla jednego wybranego algorytmu, dla jednej struktury.
Do badania pozostałych typów należy użyć: typu zmiennoprzecinkowego, typu bez znaku, łańcuchów znaków **string** (ograniczone do wartości możliwych do zapisania w pliku, tzw. *printables*).
- Ω - dla jednego wybranego algorytmu (jednego przypadku), należy porównać wszystkie trzy struktury liniowe oraz drzewo binarne i (do wyboru) kolejkę lub stos (w sumie pięć struktur).

- **Pozostałe:**

- Obiektość JEST wymagana.
- Repozytorium JEST wymagane.
- Szablony SA wymagane.

W razie pytań, pozostają do dyspozycji.